

# AIKDAP

AI Knowledge Discovery and Analytics Platform - a research platform whose answers are grounded by backend enforcement rather than by prompt instruction.

*Reference document for the research paper. All figures measured from the repository, 4 September 2026.*

---

AIKDAP answers the question that actually blocks a researcher: **is this evidence enough for my goal, and what exactly is missing?** It ingests a project's papers, retrieves against them in three stages, and produces a sufficiency verdict with a classified list of gaps - never a claim the evidence does not support.

## 1. What we used to build it

### Backend

Python 3.12 with FastAPI 0.116, async throughout, and SQLAlchemy 2.0 with typed models; every schema change goes through Alembic. Seven feature modules share one shape - **router / service / schemas / models / repository** - so business logic never sits inside a route handler.

### Storage

PostgreSQL 17 with the **pgvector** extension holds both the relational rows and the embeddings. That single choice matters more than it looks: ownership filters and vector similarity execute in one query, so the reranker can only ever see chunks the caller already owns - there is no code path where unrestricted results are filtered afterwards. Redis 8 serves three roles: Celery broker, cache, and shared LLM provider-health state. Celery 5.5 runs ingestion and research runs in the background; the API returns 202 and the run is tracked to completion.

### Orchestration

LangGraph 0.6.5 drives a six-node graph: planner -> router -> asset\_retrieval / web\_research -> context\_builder -> synthesis. Agents never call each other. A node's only output is a partial update to a shared typed state, and the graph decides what runs next. Every node is wrapped in instrumentation that times it and records success or failure, because a LangGraph exception otherwise surfaces without saying which node raised it.

### Model access

LiteLLM 1.80, imported in exactly one module - a test enforces that containment across the whole package. Around it sit retry with jittered backoff, a provider fallback chain (Gemini -> Gemini -> Groq -> OpenRouter), and a Redis-backed circuit breaker so the API and the workers share one view of an outage.

### Document ingestion

Entirely deterministic, with no model involved: pypdf, python-docx, python-pptx, openpyxl, BeautifulSoup with lxml. Scanned pages go through **Tesseract**, which also corrects 90/180/270-degree page rotation automatically. Tesseract was chosen over EasyOCR and PaddleOCR, each of which pulls a multi-gigabyte PyTorch or Paddle stack, and over PyMuPDF, whose AGPL licence is a real risk if the project ever goes commercial.

### AI models

| Role | Model | Where it runs |
|------|-------|---------------|
|------|-------|---------------|

|                        |                                               |                        |
|------------------------|-----------------------------------------------|------------------------|
| Document understanding | Qwen 3.5 4B                                   | Local, via Ollama      |
| Embedding              | BGE-M3, 1024-d                                | Local, via Ollama      |
| Reranking              | BGE-reranker-v2-m3                            | Local, via llama.cpp   |
| Planning and synthesis | Gemini, with Groq and OpenRouter as fallbacks | Cloud, via the gateway |

**Only synthesis leaves the machine.** Ingestion, understanding, embedding, retrieval and reranking are entirely local - simultaneously a cost argument, a privacy argument and an availability argument.

**Deterministic mathematics**

SymPy 1.13 parses equations and derives constraints. This never routes through an LLM, by design.

**Frontend and deployment**

React with TypeScript and Vite, Tailwind CSS, shadcn/ui and TanStack Query. Deployed with Docker Compose; JWT authentication on HS256, with a 30-minute access token and a 7-day refresh token.

**Scale**

| Measure                      | Value                                        |
|------------------------------|----------------------------------------------|
| Backend application code     | 17,792 lines across 92 Python modules        |
| Backend tests                | 11,509 lines, 528 test functions, 34 modules |
| Frontend                     | 16,802 lines across 127 files, 28 test files |
| Test-to-code ratio (backend) | about 0.65 to 1                              |
| REST endpoints               | 41, across 7 feature modules                 |

## 2. How it helps users

The user is a researcher holding a goal and a pile of papers. AIKDAP answers what actually blocks them: is this enough, and what is missing?

- **It gives a verdict, not a summary.** SUFFICIENT, PARTIALLY\_SUFFICIENT, INSUFFICIENT or AMBIGUOUS against the researcher's own stated goal, with a reason - plus a list of what is missing, each item classified REQUIRED, HELPFUL, OPTIONAL or AMBIGUOUS. A summary tells you what a paper says; a verdict tells you what to do next.
- **Every claim is checkable in one click.** Citations resolve to an exact page of an exact uploaded file, carrying the scores that caused that passage to be selected.
- **It says "I don't know."** When evidence fails the calibrated threshold, the system reports insufficient evidence and makes no generation call at all. The failure mode is silence, not a confident fabrication.
- **It surfaces contradictions instead of averaging them.** If Table 3 reports 0.82 and the abstract reports 0.86, that conflict is logged, not quietly reconciled.
- **Work accumulates.** The unit of memory is the project, not the conversation. Assets, chunks, runs, citations and experiment plans persist; nothing has to be re-uploaded or re-explained.
- **It keeps working when things break.** Reranker down, and it returns retrieval-only order and says so. Provider quota exhausted, and it moves to the next provider and records that it fell back. Local model down, and the document's text and chunks are still safely extracted.

## 3. How it differs from other tools

Every comparable system - ChatGPT, Perplexity, NotebookLM, Elicit - asks a model in a prompt to stay grounded. A prompt is a request, not a guarantee. Four concrete differences follow.

### Grounding is enforced, not requested

Every citation identifier the model returns is validated against the identifiers actually placed in its prompt. Unknown identifiers are dropped and recorded, never repaired into something plausible. The grounding status is computed by the backend from the validated set - the model does not grade itself.

### Relevance is a measured decision, not a top-k side effect

Ordering and relevance are different questions. Asked for five results, every other system returns five, however poor. AIKDAP adds a third stage that decides whether any of them are relevant at all, at a threshold calibrated on the deployment's own corpus - precision 0.818 and recall 0.818 at -2.0 - and it will return zero.

### Explainability is a schema, not a narration

Three tables - research\_runs, research\_steps and agent\_messages - reconstruct any run node by node, with timings, the plan, and the inputs that make it reproducible. Not the model's account of what it did; the record of what happened.

### It runs locally

Retrieval, embedding, understanding and OCR need no API key and cost nothing per call. That is what makes a three-pass retrieve, rerank and gate pipeline affordable at all - three model passes per query would be prohibitive against a metered API.

At the product level the gap is simpler still: nobody else answers **"is my evidence sufficient for my goal, and what exactly is missing?"**

## 4. Advantages of the platform

### Technical

- Hallucination resistance is verifiable by construction: an identifier never placed in the prompt cannot appear in a stored citation.
- Zero evidence is a calibrated, first-class outcome rather than a failure.
- Provenance survives the whole pipeline, from extracted page to stored citation, with both selection scores kept on their native scales and never fused into an invented confidence value.
- Every run is fully reconstructable from three tables.
- Failure is contained and declared at every layer - reranker, provider, local model, OCR and per-node agent failure each have a defined, reported degraded path.
- The entire pipeline except synthesis runs offline, with no API key and no per-call cost.
- Extraction, chunking, equation parsing and constraint enforcement are deterministic by design and never route through a language model.

### Architectural

- One LiteLLM boundary, enforced by a test, so adding a provider changes a single file.
- A uniform module shape across all seven feature modules.
- A stateless compiled graph with injected dependencies, safe to share across concurrent runs, with reducers already in place for future parallel fan-out.
- 528 backend tests at roughly a 0.65-to-1 test-to-code ratio, including tests that guard architectural properties - LiteLLM containment, docs exposure, execution policy - and not merely behaviour.
- Node names persist as plain strings, so extending the graph never requires a database migration.

### Product

- Project-scoped memory: work accumulates instead of evaporating with a conversation.
- A verified substrate that new deliverables can consume without touching the retrieval or grounding layers.
- Safety by boundary - the experiment engine deliberately never runs generated code, trains a model, or reports an observed result. It classifies, validates and organises, and the researcher reviews before anything executes.

---

*A note for the write-up: present the execution boundary as a design decision rather than a limitation. A system that silently fabricates an experimental result is precisely the failure mode a grounding paper exists to prevent.*